

DESIGN AND DOCUMENT THE INSTRUCTION FORMAT SPECIFICATION FOR A TWO-PASS ASSEMBLER TARGETING THE 32-BIT x86 (IA-32) ARCHITECTURE

Excluding Instruction Prefixes

Assignment Report

Course: System Software / Assembler

Architecture: 32-bit x86 (IA-32)

Object Format Used for Verification: ELF32

Submitted By: _____

Roll No.: _____

Batch / Division: _____

Academic Year: 2026

Table of Contents

1. Introduction
2. Objectives
3. IA-32 Instruction Format
4. Opcode Table
5. 32-bit Register Encoding
6. ModR/M Addressing Byte
7. SIB Addressing Byte
8. Immediate Constants and Displacements
9. Relative JMP and JNE Encoding
10. Assembler Directives: DB, DW, DD
11. SECTION .text and SECTION .data
12. Two-Pass Assembler Design
13. Practical Implementation and Verification
14. Sample Encoding Walkthroughs
15. Complete Encoding Verification Table
16. Conclusion
17. References

1. Introduction

An assembler translates assembly language instructions into machine code that can be executed by a processor. This assignment specifies how a two-pass assembler can parse and encode core 32-bit x86 (IA-32) instructions, addressing modes, immediate values, displacements, relative branches, and common data directives. The encoding is described without instruction prefixes. Verification is performed using NASM, the GNU linker, objdump, readelf, and an actual 32-bit ELF executable on Linux.

2. Objectives

- Specify the variable-length IA-32 instruction format.
- Explain opcode selection and operand encoding.
- Define complete ModR/M and SIB bit fields.
- Explain immediate constants, displacements, and Little-Endian representation.
- Explain EIP-relative displacement calculation for JMP and JNE.
- Specify DB, DW, DD and .text/.data handling.
- Describe Pass 1 and Pass 2 of a two-pass assembler.
- Verify the specified encodings with actual IA-32 machine-code output.

3. IA-32 Instruction Format

```
+-----+-----+-----+-----+-----+
| Opcode | ModR/M | SIB | Displacement | Immediate |
+-----+-----+-----+-----+-----+
| Required | Optional | Opt. | Optional | Optional |
+-----+-----+-----+-----+-----+
```

An IA-32 instruction is variable length. Depending on the instruction and addressing mode, it may contain only an opcode or may contain Opcode, ModR/M, SIB, displacement and immediate fields. In the byte stream the fields occur in the order Opcode → ModR/M → SIB → Displacement → Immediate. Multi-byte numeric values are stored in Little-Endian byte order.

4. Opcode Table

Instruction	Operand form	Encoding
MOV	r32, r/m32	8B /r
MOV	r/m32, r32	89 /r
MOV	EAX, imm32	B8+rd id
ADD	r/m32, r32	01 /r
ADD	r32, r/m32	03 /r
ADD	EAX, imm32	05 id
ADD	r/m32, imm8	83 /0 ib
SUB	r/m32, r32	29 /r
CMP	r/m32, r32	39 /r
CMP	r/m32, imm8	83 /7 ib
JMP	rel8	EB cb
JMP	rel32	E9 cd

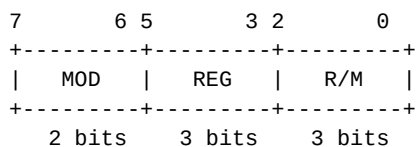
Instruction	Operand form	Encoding
JNE	rel8	75 cb
JNE	rel32	0F 85 cd
JE	rel8	74 cb
NOP	—	90
RET	—	C3
PUSH	r32	50+rd
POP	r32	58+rd

Notation: /r means that a ModR/M byte is required; ib is an 8-bit immediate; id is a 32-bit immediate; cb is an 8-bit relative displacement; cd is a 32-bit relative displacement.

5. 32-bit Register Encoding

Register	Code
EAX	000
ECX	001
EDX	010
EBX	011
ESP	100
EBP	101
ESI	110
EDI	111

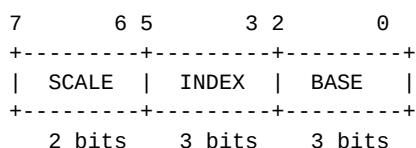
6. ModR/M Addressing Byte



$$\text{ModR/M} = (\text{MOD} \ll 6) \mid (\text{REG} \ll 3) \mid \text{R/M}$$

MOD=00 selects memory addressing normally without displacement; in 32-bit addressing R/M=101 is a special disp32 case. MOD=01 selects memory plus an 8-bit displacement. MOD=10 selects memory plus a 32-bit displacement. MOD=11 selects register-direct operands.

7. SIB Addressing Byte



$$\text{SIB} = (\text{SCALE} \ll 6) \mid (\text{INDEX} \ll 3) \mid \text{BASE}$$

SCALE	Meaning
00	×1
01	×2
10	×4

SCALE	Meaning
11	×8

INDEX and BASE use the same 3-bit register codes. A SIB byte is required when R/M=100 in 32-bit addressing (with the usual addressing-mode exceptions).

8. Immediate Constants and Displacements

Immediate constants are encoded directly after the addressing fields when required by the opcode. An immediate may be 8 or 32 bits depending on the instruction. Memory operands may use disp8 or disp32 depending on MOD. Signed disp8 values are sign-extended by the processor where applicable. Multi-byte immediates and displacements are stored Little-Endian.

```
12345678h as imm32  -> 78 56 34 12
1234h as disp16/data -> 34 12
10h as disp8        -> 10
```

9. Relative JMP and JNE Encoding

Relative branches store the displacement from the address immediately following the branch instruction. The general calculation is: relative displacement = target address – address of next instruction.

JMP practical:

```
8049000: EB 0A    jmp 804900C
Next instruction = 8049002
Target          = 804900C
rel8            = 804900C - 8049002 = 0A
Encoding        = EB 0A
```

JNE practical:

```
8049008: 75 07    jne 8049011
Next instruction = 804900A
Target          = 8049011
rel8            = 8049011 - 804900A = 07
Encoding        = 75 07
```

Short JMP uses EB cb and short JNE uses 75 cb. Near forms use E9 cd for JMP and 0F 85 cd for JNE. For rel32, the four displacement bytes are stored Little-Endian.

10. Assembler Directives: DB, DW, DD

```
section .data

num1 db 10h
num2 dw 1234h
num3 dd 12345678h
```

Actual data bytes observed with objdump:

```
10 34 12 78 56 34 12
```

DB reserves/emits 1 byte, DW emits 2 bytes, and DD emits 4 bytes. The values 1234h and 12345678h demonstrate Little-Endian ordering.

11. SECTION .text and SECTION .data

.text contains executable instructions and is normally associated with code. .data contains initialized writable data such as constants and arrays. A two-pass assembler maintains section-aware location counters so labels and data addresses can be resolved correctly.

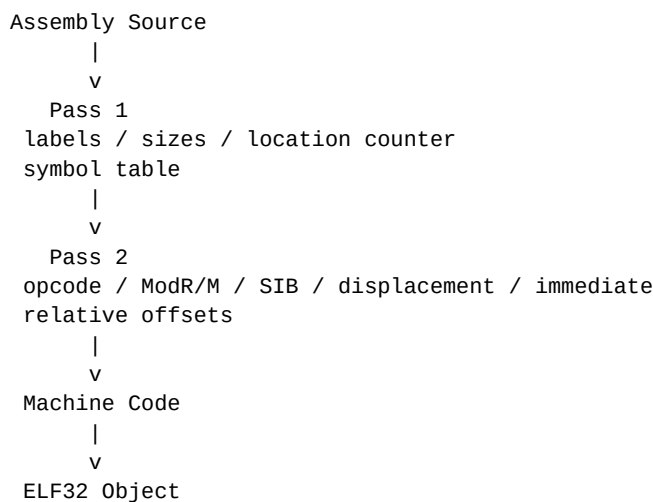
12. Two-Pass Assembler Design

Pass 1

- Read and parse source statements.
- Identify labels and directives.
- Determine instruction/data sizes.
- Maintain the location counter.
- Build the symbol table.

Pass 2

- Read the source again.
- Look up symbols in the symbol table.
- Select the opcode and operand form.
- Generate ModR/M and SIB bytes when required.
- Generate displacement and immediate fields.
- Calculate relative branch displacements.
- Emit final machine code/object output.



13. Practical Implementation and Verification

The following verification was performed on Linux using NASM for IA-32 assembly. The object file was confirmed as ELF 32-bit LSB relocatable Intel i386, and the executable was linked as `elf_i386`.

```
mkdir -p ~/IA32_Assignment
cd ~/IA32_Assignment

nasm -f elf32 test.asm -o test.o
file test.o
ld -m elf_i386 test.o -o test
file test
objdump -d -Mintel test
./test
echo $?
```

Observed object-file identification: ELF 32-bit LSB relocatable, Intel i386. The verified test program exited with status 15 after computing $5 + 10$.

14. Sample Encoding Walkthroughs

Important: The assignment wording refers to a specific “following code snippet”, but that snippet was not included in the assignment text available for this report. Therefore, the verified IA-32 examples below are used as practical encoding walkthroughs. If the teacher provides a separate mandatory snippet, that exact snippet should replace or supplement these examples.

14.1 MOV EAX, 5

```
Assembly:  mov eax, 5
Machine:   B8 05 00 00 00
```

```
B8 = MOV EAX, imm32
05 00 00 00 = 00000005h in Little-Endian
```

14.2 ADD EAX, EBX

```
Assembly:  add eax, ebx
Machine:   01 D8
```

```
01 = ADD r/m32, r32
D8 = 11011000
MOD=11, REG=011 (EBX), R/M=000 (EAX)
```

14.3 MOV ECX, [EBX + ESI*4 + 10h]

```
Assembly:  mov ecx, [ebx + esi*4 + 10h]
Machine:   8B 4C B3 10
```

```
8B = MOV r32, r/m32
4C = 01001100
    MOD=01 -> memory + disp8
    REG=001 -> ECX
    R/M=100 -> SIB follows
```

```
B3 = 10110011
    SCALE=10 -> x4
    INDEX=110 -> ESI
    BASE =011 -> EBX
```

```
10 = disp8 = 10h
```

```
Final: 8B 4C B3 10
```

This is the key complex-addressing example: [EBX + ESI*4 + 10h] is represented by a ModR/M byte selecting SIB addressing, followed by a SIB byte and an 8-bit displacement.

15. Complete Encoding Verification Table

Assembly	Machine code	Main concept
MOV EAX,5	B8 05 00 00 00	Opcode + imm32
MOV EBX,10	BB 0A 00 00 00	Opcode + imm32
ADD EAX,EBX	01 D8	ModR/M
MOV EBX,EAX	89 C3	ModR/M
CMP EAX,5	83 F8 05	ModR/M + imm8
JMP target	EB 0A	Relative rel8
JNE not_equal	75 07	Conditional rel8
MOV ECX,[EBX+ESI*4+10h]	8B 4C B3 10	ModR/M + SIB + disp8
DB 10h	10	DB

Assembly	Machine code	Main concept
DW 1234h	34 12	DW + Little-Endian
DD 12345678h	78 56 34 12	DD + Little-Endian

16. Conclusion

The IA-32 instruction format is variable length and can combine opcode, ModR/M, SIB, displacement and immediate fields. The ModR/M byte identifies operand registers and addressing modes, while the SIB byte represents scaled-index addressing. Immediate values and displacements follow the instruction encoding rules and multi-byte quantities use Little-Endian byte order. Relative branches use a displacement measured from the next instruction. A two-pass assembler can resolve labels and sizes in Pass 1 and generate final machine-code fields in Pass 2. The specification was practically verified using NASM and IA-32 ELF32 tools on Linux.

17. References

- Intel 64 and IA-32 Architectures Software Developer's Manual, Volume 2: Instruction Set Reference.
- Intel 64 and IA-32 Architectures Software Developer's Manual, Volume 1: Basic Architecture.
- NASM documentation and NASM x86 instruction encoding reference.
- GNU binutils documentation: objdump, readelf and ld.